

## TP 29 — Gloutons au banc d'essai

Semaine 29 ■ Fichiers dans NSI/TP29/ ■ 2 h

### Objectifs

- Implémenter rendu de monnaie et sac à dos gloutons
- Chasser les contre-exemples par programme
- Comparer glouton et exploration exhaustive (petites tailles)

### Partie A — Rendu de monnaie — `monnaie.py`

(35 min)

1. Implémenter `rendu(somme, pieces)` (cours) avec docstring et tests (dont `rendu(0, ...) → []`).
2. `nb_pieces(somme, pieces)` — juste le nombre (réutiliser).
3. Vérifier l'euro : afficher `nb_pieces(s, EURO)` pour `s` de 1 à 20.
4. **La chasse au contre-exemple** : pour le système `[5, 4, 1]`, écrire `optimal_force_brute(somme)` qui essaie toutes les combinaisons raisonnables (trois boucles imbriquées sur le nombre de 5, de 4, de 1 — bornées par `somme`) et renvoie le minimum de pièces. Afficher les sommes de 1 à 30 où le glouton fait strictement pire.

### Partie B — Sac à dos — `sac.py`

(45 min)

Les objets : `{"nom": ..., "masse": ..., "valeur": ...}`.

1. `valeur_totale(choix)` et `masse_totale(choix)` (sommes sur un champ — les yeux fermés désormais).
2. Les trois gloutons : `glouton_valeur`, `glouton_masse`, `glouton_rapport` — même squelette, seule la **clé de tri** change (en faire un paramètre ! `sac(objets, capacite, cle, sens)`).
3. Sur le jeu d'objets de l'exercice 3, vérifier que les trois ratent l'optimum (40).
4. Générer 20 objets aléatoires (masse 1-10, valeur 1-50), capacité 30 : comparer les trois gloutons. Le rapport valeur/masse gagne-t-il toujours ? (relancer plusieurs fois)

### Partie C — Le juge exhaustif ★★

(25 min)

Pour  $n \leq 15$  objets, on peut tout essayer : chaque sous-ensemble correspond à un mot binaire de  $n$  bits (semaine 5 : les voilà encore).

```
def optimal(objets, capacite):
    """Renvoie la meilleure valeur possible (force brute 2**n)."""
    n = len(objets)
    meilleur = 0
    for code in range(2 ** n):
        masse = 0
        valeur = 0
        for i in range(n):
            if (code // 2 ** i) % 2 == 1:  # bit i allumé ?
                masse = masse + objets[i]["masse"]
                valeur = valeur + objets[i]["valeur"]
        if masse <= capacite and valeur > meilleur:
            meilleur = valeur
    return meilleur
```

1. Comprendre puis tester sur l'exemple à 4 objets (attendu : 40).
2. Sur 12 objets aléatoires : écart moyen du meilleur glouton à l'optimum (10 tirages) ?
3. Chronométrer **optimal** pour  $n = 10, 15, 20$  (20 : patienter...). Combien de temps pour  $n = 40$  ? (calculer, pas essayer !)

### Partie D — Défi ★★

(15 min)

Le glouton du plombier : des tuyaux de longueurs données, des commandes à servir ; ou plus simple — **le remplissage de cartons** : des colis de masses diverses, des cartons de 10 kg max, minimiser le nombre de cartons. Glouton proposé : trier les colis par masse décroissante, placer chacun dans le premier carton où il tient. Implémenter et tester sur  $[7, 6, 5, 4, 3, 3, 2]$  (combien de cartons ?).